

Fractal Forge — Full Final Script Draft

Part 1 — What Is a Fractal?

We begin with the basic question:

What is a fractal?

A fractal is a structure generated by repeatedly applying a simple rule.

This rule can be geometric, meaning it is based on manipulating a shape, or it can be formula-based, meaning it is based on repeatedly applying a mathematical equation.

The important idea is that fractals are practical examples of iteration.

Iteration means repeating the same process again and again.

At first, the rule may look simple. But after repeating it many times, the result can become highly detailed and complex.

To understand this, let us start with a simple geometric example: the Sierpinski triangle.

We begin by drawing one equilateral triangle.

Then we mark the midpoint of each side.

After that, we connect the three midpoints together.

This divides the original triangle into four smaller equal triangles.

Now, we remove or change the color of the middle triangle so that only the three corner triangles remain.

Then, for each of the three remaining corner triangles, we repeat the same process.

We mark the midpoints, connect them, and remove the middle triangle again.

If we continue repeating this rule, the shape approaches the famous Sierpinski triangle.

The strange thing is that the rule is simple, but the final result looks complex.

This is one of the main ideas behind fractals:

A simple repeated rule can create a complex structure.

Website interaction idea:

In the website, this part can be shown using a simple animation. The user starts with one triangle, then presses “next step” or moves a slider to increase the number of iterations. With every step, the Sierpinski triangle becomes more detailed.

Part 2 — Geometric Fractals and Formula-Based Fractals

The Sierpinski triangle is an example of a geometric fractal.

A geometric fractal is created by directly changing a shape using a repeated geometric rule.

For example, we divide a shape, remove part of it, copy part of it, or replace part of it with a smaller version.

But geometric construction is not the only way to create fractals.

Another important type is formula-based fractals.

In formula-based fractals, we do not start by directly drawing a shape.

Instead, we start with a value, usually a number or a point on a plane.

Then we plug this value into a formula.

The result is plugged back into the same formula.

This process repeats again and again.

In simple form, we can write it like this:

new value = formula(old value)

After many repetitions, we observe the behavior of the values.

Some values may settle down.

Some may escape and grow without limit.

Some may move toward a specific point.

Some may remain bounded and create complicated patterns.

The fractal appears when we color points based on their behavior.

This is different from geometric fractals.

In geometric fractals, the rule acts directly on a shape.

In formula-based fractals, the rule acts on numbers first, and the shape appears after we visualize the behavior of those numbers.

In our project, we use both types.

For geometric fractals, we use examples such as the Sierpinski triangle and the Koch snowflake.

For formula-based fractals, we use examples such as Julia sets, the Mandelbrot set, Newton's fractal, and we may also mention Nova fractals as another possible formula-based fractal family.

This gives Fractal Forge two sides:

One side shows how repeated geometric construction creates fractals.

The other side shows how repeated formulas create fractals.

Website interaction idea:

This section can show two cards side by side.

The first card says "Geometric Fractal" and shows a Koch snowflake or Sierpinski triangle being built step by step.

The second card says "Formula-Based Fractal" and shows a formula being iterated, then converted into a visual image.

Part 3 — Koch Snowflake

Now we move to another important geometric fractal: the Koch snowflake.

The Koch snowflake starts with an equilateral triangle.

Then we apply the same rule to every side.

The rule is:

First, divide each side into three equal parts.

Second, remove the middle third.

Third, replace that middle third with the two sides of a small outward equilateral triangle.

This means that every straight line segment is replaced by four smaller line segments.

Then we repeat the same process on every new segment.

At each iteration, the shape becomes more detailed.

The Koch snowflake is important because it shows a very unusual property:

It has an infinite perimeter, but it encloses a finite area.

This sounds strange at first.

In normal geometry, we usually expect that if the boundary keeps getting longer forever, the shape should also become infinitely large.

But the Koch snowflake does not do that.

The boundary keeps increasing, but the whole snowflake still fits inside a limited region. The Koch snowflake's perimeter grows without bound while its enclosed area approaches a finite limit.

Now we explain this using formulas.

Let n represent the iteration number.

Let s represent the side length of the original equilateral triangle.

At the beginning, the triangle has 3 sides.

At each iteration, every side becomes 4 smaller sides.

So the number of sides at step n is:

$$N_n = 3 \times 4^n$$

where:

N_n is the number of sides at step n .

3 is the original number of sides.

4 is the multiplication factor, because every old side becomes 4 new sides.

For example, at the first iteration:

$$N_1 = 3 \times 4 = 12$$

So after one iteration, the shape has 12 sides.

Now let us look at the length of each side.

At every iteration, each side is divided by 3.

So the length of each small side at step n is:

$$l_n = s / 3^n$$

where:

l_n is the length of each side at step n .

s is the original side length.

3^n appears because the side length is divided by 3 at each iteration.

Now, the perimeter is:

$$P_n = N_n \times l_n$$

Substituting the formulas:

$$P_n = (3 \times 4^n)(s / 3^n)$$

So:

$$P_n = 3s(4/3)^n$$

Since:

$$4/3 > 1$$

The perimeter keeps increasing as n increases.

So when the number of iterations approaches infinity:

$$\lim(n \rightarrow \infty) P_n = \infty$$

This means the Koch snowflake has an infinite perimeter.

But its area does not become infinite.

The added triangles become smaller and smaller, so the total area approaches a fixed finite value.

This is related to the coastline paradox.

The more precisely we measure a rough coastline, the longer it appears, because we capture smaller and smaller details.

The Koch snowflake is a mathematical version of this idea.

Website interaction idea:

The website can show a Koch snowflake with an iteration slider.

When the user increases the iteration number, the shape updates.

Next to it, the website shows:

Number of sides: increasing.

Side length: decreasing.

Perimeter: increasing.

Area: approaching a fixed limit.

This makes the paradox visible.

Part 4 — Fractal Dimension

After seeing the Koch snowflake, we now need a way to describe how much space a fractal fills.

In normal geometry, we usually describe dimension in a simple way.

A line is one-dimensional because it extends in one direction.

A square is two-dimensional because it extends horizontally and vertically.

A cube is three-dimensional because it extends in three directions: length, width, and height.

This is the conventional idea of dimension.

But fractals do not always fit perfectly into these categories.

For example, the Koch curve is not a filled area like a square.

So it is not fully two-dimensional.

But it is also more complex than a normal line.

It bends, branches, and fills more space than a simple one-dimensional line.

So its dimension is between 1 and 2.

This is called a non-integer dimension, or fractal dimension.

Fractal dimension does not mean that the shape exists in a strange physical axis.

Instead, it measures how the shape fills space as we zoom in.

To explain this, we use the self-similarity dimension formula:

$$N = (1/r)^D$$

where:

N is the number of self-similar copies.

r is the scale factor of each copy.

D is the fractal dimension.

Solving for D , we get:

$$D = \log(N) / \log(1/r)$$

Let us test this formula on familiar shapes.

For a line, if we divide it into 2 equal smaller copies, each copy has scale factor:

$$r = 1/2$$

and the number of copies is:

$$N = 2$$

So:

$$2 = 2^D$$

Therefore:

$$D = 1$$

This matches what we know: a line is one-dimensional.

For a square, if we divide it into 4 equal smaller squares, each copy has scale factor:

$$r = 1/2$$

and:

$$N = 4$$

So:

$$4 = 2^D$$

Therefore:

$$D = 2$$

This also matches what we know: a square is two-dimensional.

Now let us apply this to the Koch curve.

In the Koch curve, every segment is replaced by 4 smaller segments.

So:

$$N = 4$$

Each small segment is one third of the original length.

So:

$$r = 1/3$$

Using the formula:

$$D = \log(4) / \log(3)$$

This gives approximately:

$$D \approx 1.262$$

So the Koch curve has a dimension greater than 1, but less than 2.

This means it is more complex than a line, but it does not fully fill an area like a square.

For the Koch snowflake, we must be careful.

The boundary of the snowflake behaves like the Koch curve, so the boundary has a fractal dimension around 1.262.

But the filled region inside the snowflake is still a two-dimensional area.

So we can say:

The boundary is fractal.

The filled snowflake region is two-dimensional.

This distinction is important because it shows that fractals can contain different dimensional behaviors in the same object.

Website interaction idea:

This part can compare three objects:

A line with dimension 1.

A square with dimension 2.

A Koch curve with dimension approximately 1.262.

The website can show how each object changes when zoomed or subdivided.

This helps the user understand that fractal dimension measures how strongly a pattern fills space.

Part 5 — Transition to Formula-Based Fractals

Until now, we have focused mostly on geometric fractals.

We saw how a shape can become complex when a simple construction rule is repeated.

Now we move to formula-based fractals.

Here, the rule is not applied directly to a visible shape.

Instead, the rule is applied to numbers.

Then we visualize the result by coloring points based on their behavior after many iterations.

This is where fractals become a way to visualize mathematical behavior.

To understand this, we first need the complex plane.

Part 6 — Complex Numbers and the Complex Plane

A complex number is written in the form:

$$z = a + bi$$

where:

a is the real part.

b is the imaginary part.

i is the imaginary unit, where:

$$i^2 = -1$$

Complex numbers may look abstract, but visually they are very useful.

A complex number can be represented as a point on a plane.

The horizontal axis represents the real part.

The vertical axis represents the imaginary part.

This plane is called the complex plane.

For example:

$$z = 2 + 3i$$

means the point has real value 2 and imaginary value 3.

This is important for fractals because each pixel on the screen can represent one complex number.

Then we apply a formula to that number repeatedly.

After many iterations, we color that pixel depending on what happened.

Did the value stay bounded?

Did it escape?

Did it move toward a root?

The image appears from the behavior of numbers.

This is what makes formula-based fractals powerful.

They transform invisible numerical behavior into visible patterns.

Website interaction idea:

The website can show a complex plane.

When the user moves the mouse over the plane, the website displays the complex number at that point.

For example:

$$z = 0.4 + 0.2i$$

This prepares the user for Julia sets, Mandelbrot sets, and Newton's fractal.

Part 7 — Julia Set

The first formula-based fractal we explain is the Julia set.

The Julia set is created using the repeated formula:

$$z(n+1) = z(n)^2 + c$$

where:

$z(n)$ is the current complex number.

$z(n+1)$ is the next complex number after applying the formula.

c is a fixed complex constant.

The main idea is this:

In a Julia set, we choose one fixed value of c , then we test many different starting values of z .

Each point on the screen represents a different starting value of z .

For each point, we apply the formula repeatedly:

$$z_0 \rightarrow z_1 \rightarrow z_2 \rightarrow z_3 \rightarrow \dots$$

Then we observe what happens.

If the values stay bounded, the point belongs to the filled Julia set.

If the values escape to infinity, the point is outside the set.

The final image is created by coloring the points.

Points that stay bounded may be given one color.

Points that escape may be colored depending on how quickly they escape.

This is called escape-time coloring.

The interesting thing about Julia sets is that the value of c controls the shape.

A small change in c can create a completely different Julia set.

One value may create a connected shape.

Another value may create scattered dust-like points.

Another value may create spirals or branch-like structures.

So the Julia set shows how one simple formula can create many different visual worlds.

Website interaction idea:

The Julia section can include sliders for the real and imaginary parts of c .

When the user changes c , the Julia set updates.

This lets the user see that c controls the personality of the Julia set.

Part 8 — Mandelbrot Set

After the Julia set, we move to the Mandelbrot set.

The Mandelbrot set uses the same formula:

$$z(n+1) = z(n)^2 + c$$

But the role of the variables changes.

In the Julia set, c is fixed and z changes.

In the Mandelbrot set, we always start with:

$$z_0 = 0$$

and we allow c to change.

Each point on the screen represents a different value of c .

For every value of c , we start at $z_0 = 0$, then repeat:

$$z(n+1) = z(n)^2 + c$$

If the sequence stays bounded, then that point c belongs to the Mandelbrot set.

If the sequence escapes to infinity, then that point is outside the Mandelbrot set. The Mandelbrot set is defined as the set of complex numbers c for which the iteration $z(n+1) = z(n)^2 + c$, starting from $z = 0$, remains bounded.

The Mandelbrot set is famous because its boundary contains endless detail.

No matter how much we zoom in, new structures continue to appear.

Some parts look similar to the whole set.

Other parts reveal completely new patterns.

This makes the Mandelbrot set one of the strongest examples of complexity from a simple rule.

But the Mandelbrot set is not only a beautiful image.

It also has a deeper meaning.

It is a map of possible behaviors.

Each point in the Mandelbrot set represents a value of c .

That same value of c can be used to generate a Julia set.

So the Mandelbrot set can be understood as a map of Julia sets.

Website interaction idea:

The website can show the Mandelbrot set on one side and a Julia set on the other side.

When the user clicks a point on the Mandelbrot set, that point becomes the value of c .

Then the website generates the corresponding Julia set.

This makes the relationship visual and interactive.

Part 9 — Julia Set and Mandelbrot Set Comparison

Julia sets and the Mandelbrot set are closely connected because they use the same formula:

$$z(n+1) = z(n)^2 + c$$

But they ask different questions.

The Julia set asks:

What happens to different starting points z when c is fixed?

The Mandelbrot set asks:

What happens when we start from $z_0 = 0$ and test different values of c ?

So the difference is not the formula.

The difference is what we allow to change.

In the Julia set:

c is fixed.

z changes.

Each pixel represents a starting value of z .

In the Mandelbrot set:

z_0 is fixed at 0.

c changes.

Each pixel represents a value of c .

A simple way to explain the relationship is:

The Julia set is one world created by one value of c .

The Mandelbrot set is a map of possible worlds.

Another way to say it is:

The Mandelbrot set is the control map.

The Julia set is the result.

This relationship is one of the strongest ideas in Fractal Forge.

Instead of only showing two separate fractals, we connect them.

The user can click a point in the Mandelbrot set and instantly generate the Julia set for that point.

In this way, the user does not only look at fractals.

They explore how changing one parameter changes an entire mathematical world.

Mathematically, the Mandelbrot set can also be described as the set of c values for which the corresponding Julia set is connected; outside it, the Julia set becomes disconnected or dust-like.

Website interaction idea:

This can be one of the main interactive features of the website.

The screen can be split into two panels.

Left panel: Mandelbrot set.

Right panel: Julia set.

Clicking a point on the Mandelbrot set updates the Julia set.

This creates a direct bridge between formula, parameter, and visual result.

Part 10 — Newton's Fractal

Now we move to another formula-based fractal: Newton's fractal.

Newton's fractal is based on Newton's method.

Newton's method is normally used to approximate the roots of an equation.

A root is a value that makes a function equal to zero.

For example, if:

$$f(x) = x^2 - 1$$

then the roots are:

$$x = 1$$

and:

$$x = -1$$

because both values make the function equal to zero.

Newton's method starts with an initial guess.

Then it improves that guess step by step using the formula:

$$z(n+1) = z(n) - f(z(n)) / f'(z(n))$$

where:

$z(n)$ is the current guess.

$f(z(n))$ is the value of the function at that guess.

$f'(z(n))$ is the derivative of the function.

$z(n+1)$ is the improved guess.

Normally, Newton's method is used to find one answer.

But when we apply Newton's method to many points on the complex plane, it creates a fractal.

Each point on the screen becomes a starting guess.

Newton's method is applied repeatedly to that point.

Eventually, the point may move toward one of the roots of the function.

Then we color the point based on which root it reaches.

For example, if the equation has three roots, we can use three colors.

Points that move toward the first root get one color.

Points that move toward the second root get another color.

Points that move toward the third root get another color.

The final result is Newton's fractal.

The surprising part is the boundary between the colored regions.

Two starting points can be extremely close to each other, but they may move toward different roots.

That creates a complicated fractal boundary.

Newton's fractal is usually described as a fractal in the complex plane produced by applying Newton's method to a fixed function, where different regions correspond to different roots or outcomes.

This shows that even a method designed to solve equations can create complex visual patterns.

Newton's fractal connects algebra, calculus, complex numbers, iteration, and visualization.

Website interaction idea:

The Newton fractal section can allow the user to choose a function, such as:

$$f(z) = z^3 - 1$$

The website shows the roots and the colored regions.

The user can click a starting point and watch how Newton's method moves step by step toward a root.

This makes Newton's method visual instead of only algebraic.

Part 11 — Summary

Now we connect all the fractals we discussed.

The Sierpinski triangle shows how a simple geometric rule can create a self-similar structure.

The Koch snowflake shows how repeated construction can create an infinite perimeter inside a finite area.

Fractal dimension shows that some fractals are more complex than lines but do not fully fill a plane.

The Julia set shows how fixing one value of c creates a full fractal world.

The Mandelbrot set shows how changing c creates a map of possible behaviors.

Newton's fractal shows how solving equations can create fractal boundaries.

All of these examples are different, but they share the same foundation.

They are all based on repeated rules.

They all show how simple mathematical instructions can create complex results.

They all turn abstract mathematical ideas into visual structures.

This is the purpose of Fractal Forge.

Fractal Forge is not only a website that displays fractals.

It is an interactive environment where users can see, change, and understand the rules behind fractals.

The goal is to move from passive viewing to active exploration.

The user should not only ask:

“What does this fractal look like?”

They should also ask:

“What rule created it?”

“What happens if I change the rule?”

“What happens if I change the parameter?”

“What happens if I zoom in?”

This is where the educational value of the project appears.

The project teaches mathematics through interaction.

Assisting parts

Part 13 — Website Structure and Interactive Features

The website will be divided into interactive sections.

Each section explains one idea and includes a small visual interaction.

Section 1: What is a fractal?

Interaction: Sierpinski triangle step-by-step animation.

The user can increase the number of iterations and see how a simple triangle becomes a fractal.

Section 2: Geometric vs formula-based fractals.

Interaction: Side-by-side comparison.

One side shows a geometric construction, and the other side shows a formula being iterated.

Section 3: Koch snowflake.

Interaction: Iteration slider.

The user changes the iteration number and sees the snowflake become more detailed.

The website displays the number of sides, side length, perimeter, and area behavior.

Section 4: Fractal dimension.

Interaction: Dimension comparison.

The website compares a line, a square, and a Koch curve.

This helps explain why the Koch curve has a dimension between 1 and 2.

Section 5: Complex plane.

Interaction: Mouse position reader.

When the user moves the mouse over the plane, the website displays the complex number at that position.

Section 6: Julia set.

Interaction: Parameter sliders.

The user changes the real and imaginary parts of c , and the Julia set updates.

Section 7: Mandelbrot set.

Interaction: Zoom and explore.

The user can zoom into the Mandelbrot set and see new details appear.

Section 8: Mandelbrot and Julia comparison.

Interaction: Click-to-generate Julia.

The user clicks a point in the Mandelbrot set, and the website generates the corresponding Julia set.

Section 9: Newton's fractal.

Interaction: Root attraction visualization.

The user clicks a starting point and watches it move toward a root using Newton's method.

Section 10: Experimental features.

Interaction ideas:

Image rendering and extraction.

A custom formula renderer.

Custom geometric instructions.

Possibly using AI to help convert instructions into visual fractal rules.

Sound interaction with the Mandelbrot set, where values from the fractal are converted into audio patterns.

These experimental features can make the website feel more innovative and unique.

Part 14 — Transition to the Game

After explaining and visualizing the fractals, we reach the main interactive part of the project: the game.

Until now, the user has learned the main ideas.

They have seen how fractals are made.

They have seen how iteration works.

They have seen how dimension can be non-integer.

They have seen how complex numbers allow formulas to become images.

They have explored Julia sets, the Mandelbrot set, and Newton's fractal.

But the project should not stop at explanation.

The final step is to let the user use the ideas.

That is the purpose of the game.

The game should not be random entertainment.

It should be connected to the mathematics.

The player should interact with fractal rules, parameters, iterations, patterns, or visual behavior.

The goal is to make the user understand fractals by doing something, not only by reading.

In other words:

The explanation teaches the idea.

The visualization shows the idea.

The game makes the user apply the idea.

This is what makes Fractal Forge a math innovation.

It transforms fractals from abstract mathematical objects into interactive experiences.